Problem Statement
Some people that could really benefit from blog posts or articles do not have access to them due to various factors like visual impairment, dyslexia, or concentration problems. It could also be due to non-medical issues like time constraint or convenience. This project is also useful for creators as it expands their reach. Creators can take their blog, pass it to the agent and get another version of their work that would appeal to a wider audience, making them more efficient. This agent takes a blog URL, rewrites it for audio, and uses Text-To-Speech to transform it to audio for individuals who would need it.

Option A: The Single Agent
This project could use either a single agent, or multiple agents. We could have one agent handle two tools; A web scraper, whose findings it rewrites, and a TTS afterwards for translation. A second option is to have two agents. One agent calls a web scrapper before cleaning/rewriting the text, while the second agent reviews the work of the first agent, and then calls a TTS to translate to audio. For simplicity, we'd go for a single agent with multiple tools.

Deep Learning/course module Connection:
Our agent applies agentic systems using tools, RAG and reasoning (module 10). Firstly, GPT 4-o-mini is the reasoning brain of the agent. After the blog text is scrapped, GPT 4-o-mini doesn't just copy it, it actually understands the meaning of the content, identifies what makes sense when spoken out loud versus what only works in writing and rewrites it to a natural flowing script. We used a GPT-based transformer here because rewriting for audio requires contextual understanding across the entire blog post, not just sentence by sentence. A simpler model wouldn't know that a bullet point list needs to become a flowing sentence, or that "click the link below" is meaningless in audio and needs to be cut entirely.
Secondly, OpenAI TTS; a neural sequence-to-sequence speech synthesis model. This appeals to module 4 (sequence modelling and NLP). This is not just a text reader, it's a deep learning model trained to understand the rhythm and cadence of the human voice. We chose it because the point of this project is for the output to sound human instead of just a robot reading words. A non-neural TTS would produce a flat sounding audio, which defeats the purpose. A neural model understands that a question should sound like a question and a pause should come after a major point; things that make the audio more listenable.
Together the two models cover the full pipeline; GPT 4-o-mini to understand and script rewriting, and the neural TTS model to translate that text into human-sounding audio.

Agent Framework:

We plan to use Langchain because it has a big community and great support for single-agent tool integration. I was also recommended to us for the case of a single agent like our particular project.
We considered Autogen first, but later decided against it because AutoGen's core strength is coordinating conversations between multiple agents. Since the project is a single agent with two tools, AutoGen would be like bringing a whole management team to manage one person; It adds complexity without significant value.
We also considered CrewAI, but crewAI is purpose built for multi-agent role-based systems where agents have defined personas and work in harmony to accomplish tasks. Again, that's more architecture than the project needs.
Langchain is the sweet spot. It's lightweight enough for a single agent build, has clean tool registration built in, integrates directly with OpenAI API, and has the best documentation and community for any agent framework right now, with matters for our limited timeframe.

Tools And Data
(1) BeautifulSoup (free, open-source Python library): The agent needs this to extract the actual readable text from a blog or article URL. Without it, the agent receives raw HTML full of tags, scripts, and formatting noise that GPT-4o-mini would have to wade through. BeautifulSoup strips all of that and hands back clean paragraph text, so the LLM is only working with the content that actually matters.
(2) OpenAI GPT-4o-mini via the OpenAI API: The agent needs this as its reasoning brain to rewrite the scraped text into a natural spoken script. This is the step that makes the output actually sound like a podcast and not a blog post being read aloud. Without this rewriting step, the audio output would contain bullet points, hyperlink references, and written language structures that make no sense in audio format.
(3) OpenAI TTS via the OpenAI API: The agent needs this to convert the rewritten script into a human-sounding audio file. This is the final output the user actually receives. At roughly $0.075 per average blog post, the cost per run is negligible for a project of this scale.
Data: The agent will work with publicly available blog and article posts pulled live from real URLs at runtime. For development and testing we will use 4-5 real posts from freely accessible sites like Medium, DEV.to, or personal tech blogs; these are public content with no authentication required. For the demo we will select one clean, well-structured blog post that represents a realistic use case. Since the agent reads live public web content and does not store or retain any of it, there are no privacy concerns. There are also no licensing issues because the agent is not reproducing or redistributing the content; it is transforming it into a new format for personal use, similar to how a read-aloud accessibility tool works.

Expected Challenges

Challenge 1: Some blog websites block automated scraping. BeautifulSoup works by sending an HTTP request to a URL and parsing the HTML that comes back. Some websites, particularly larger publications like Medium or Forbes detect automated requests and either block them entirely or return a login wall instead of the actual content. This means the scraper could fail silently, returning little to no usable text depending on the blog being tested. To mitigate this, We will test BeautifulSoup against multiple blog sources early in the build and identify which ones work cleanly. If a site blocks the scraper, We will add a manual text input fallback, where the user can paste the blog content directly; so the rest of the pipeline can still run and be demonstrated without depending on a live URL.

Challenge 2: GPT-4o-mini may not consistently remove all written-only language from the script. Blog posts frequently contain phrases like "as shown in the image above," "click the link below," "see the table on the right," or footnote references that are completely meaningless in an audio format. GPT-4o-mini is a strong rewriter but may not catch every instance of these written artifacts, especially in longer or more complex posts. If this happens the audio output will sound unpolished and break the listening experience. To mitigate this we will write a detailed and explicit system prompt that specifically instructs the model to remove any visual references, hyperlink language, and non-audio-friendly formatting. We will also test the rewriting step on at least five different blog posts of varying length and style before connecting it to the TTS tool, so we can refine the prompt iteratively before the final demo.

A Little Briefing:

Tools

Tool	**Purpose**
BeautifulSoup	Scrapes and cleans blog/article content from URL
GPT-4o-mini	Rewrites blog into natural spoken script
OpenAI TTS	Converts script into audio

Deep Learning Connection

Transformers- GPT-4o-mini handles the rewriting using self-attention to understand context and restructure content naturally.

RNNs- The sequence modelling intuition applies to how the rewriting agent processes the blog paragraph by paragraph, maintaining flow and context throughout.

CNNs- The datatype here is text, not images

Anticipated Challenges

Scraping Blocked Sites- Some blogs block scrapers. Mitigation: Allow pasting raw blog text

Rewrite Quality- GPT-4o-mini might occasionally leave in written artifacts like “Click the link below.” Mitigation would be to add explicit instructions in the system prompt to strip away anything that wouldn’t make sense in the output.

Blog/Audio Length- Long blog posts could hit TTS limits or cost more. Mitigation would be to add a word count check and trim or summarize anything over a set limit before sending to TTS